Materia: Programación IV

UNIDAD 7 · SERVER STATE Y NAVEGACIÓN

APUNTE TEÓRICO

Guía Teórica: TanStack Query y el Manejo de Server State

Revisor de la unidad: Prof. Franco González

1. Introducción: el problema del frontend moderno

En las aplicaciones de React que obtienen datos con `useEffect` y `fetch`, aparecen problemas que se agravan a medida que la aplicación crece:

- **Código repetitivo:** cada componente debe replicar la lógica de estados de carga, error y datos.
- **Ausencia de caché:** al desmontar y montar un componente, se dispara una nueva petición innecesaria.
- **Solicitudes duplicadas:** varios componentes pueden pedir los mismos datos de forma simultánea, lo que genera tráfico de red innecesario.
- **Sincronización manual:** mantener los datos actualizados frente a cambios en el servidor requiere lógica adicional (intervalos de actualización, detección del foco de la ventana, invalidaciones manuales).

2. ¿Qué es TanStack Query?

TanStack Query es una biblioteca diseñada específicamente para gestionar el **Server State** (Estado del Servidor). Se encarga de abstraer toda la complejidad del ciclo de vida de los datos remotos.

Diferencias entre Client State y Server State

Concepto	Client State	Server State
Definición	Estado interno y local de la UI.	Datos que residen en servidores o APIs externas.
Ejemplos	Modales, formularios, tema claro u oscuro.	Catálogo de productos, perfiles de usuario.
Desafío	Es fácil de controlar con `useState`.	Puede quedar desactualizado y requiere sincronización.

Concepto	Client State	Server State
Herramienta	`useState`, `useReducer`, Zustand, Redux.	TanStack Query, SWR, RTK Query.

3. Configuración del entorno

Para implementar TanStack Query, se siguen tres pasos fundamentales:

1. **Instalación:** `pnpm add @tanstack/react-query`.
2. **Creación del cliente:** Se instancia un `QueryClient` para gestionar la caché global.
3. **Provider:** Se envuelve la aplicación con `QueryClientProvider` para dar acceso al contexto en todo el árbol de componentes.

El siguiente código aplica los pasos 2 y 3 en el archivo de entrada de la aplicación:

```tsx
// main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import App from "./App";

const queryClient = new QueryClient();

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </StrictMode>
);
```

El `QueryClient` se crea una sola vez, fuera de los componentes, para que la caché no se reinicie en cada renderizado. Todos los componentes que quedan dentro del `QueryClientProvider` pueden usar los hooks de la biblioteca.

4. Implementación técnica (código)

A. Definición de tipos y acciones (API)

Es fundamental tipar los datos para obtener autocompletado y detectar errores en tiempo de compilación.

```ts
// types/product.ts
export interface Product {
  id: number;
  title: string;
  price: number;
  description: string;
  category: string;
  image: string;
}

// api/products.actions.ts
import type { Product } from "../types/product";
export const getProducts = async (): Promise<Product[]> => {
  const res = await fetch("https://fakestoreapi.com/products");

  // fetch no rechaza la promesa ante respuestas HTTP de error (404, 500):
  // se lanza un error para que la query pase al estado de error (isError)
  if (!res.ok) {
    throw new Error("Error al cargar productos");
  }

  return res.json();
};
```

B. Uso del hook `useQuery`

El hook `useQuery` es la herramienta principal para leer datos. Requiere una `queryKey` (identificador único para la caché) y una `queryFn` (la función que hace la petición).

```tsx
// pages/ProductsPage.tsx
import { useQuery } from "@tanstack/react-query";
import { getProducts } from "../api/products.actions";
import type { Product } from "../types/product";

export const ProductsPage = () => {
  // El hook devuelve estados automáticos: data, isLoading, isError
  const { data, isLoading, isError } = useQuery<Product[]>({
    queryKey: ["products"],
    queryFn: getProducts,
  });

  if (isLoading) return <p>Cargando...</p>;
  if (isError) return <p>Error al cargar datos</p>;
  if (!data || data.length === 0) return <p>No hay productos para mostrar.</p>;

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Productos</h1>
      <div className="grid grid-cols-3 gap-6">
        {data?.map((product) => (
          <div key={product.id} className="shadow-md p-5 rounded-2xl">
            <h2 className="font-semibold">{product.title}</h2>
          </div>
        ))}
      </div>
    </div>
  );
};
```

En TanStack Query v5, `isPending` indica que la query todavía no tiene datos, mientras que `isLoading` es verdadero cuando la query no tiene datos en caché y además hay una petición en curso (equivale a `isPending && isFetching`); ocurre en la primera carga y también, por ejemplo, cuando cambia la `queryKey` a una clave sin datos. En este componente, ambas banderas cubren el estado de carga inicial. Además, una query que falla se reintenta por defecto tres veces, con una espera creciente entre intentos, antes de pasar al estado de error; por eso el mensaje de error puede tardar unos segundos en aparecer. El número de reintentos se configura con la opción `retry`.

C. Query keys, staleTime y gcTime

La `queryKey` es un arreglo serializable que identifica de forma única los datos de una query en la caché. Los componentes que usan la misma clave comparten los mismos datos, y esa misma clave se utiliza después para invalidar la query y forzar una nueva petición. Si la `queryFn` depende de una variable, esa variable forma parte de la clave, por ejemplo `["products", id]`: cuando la variable cambia, la query se ejecuta de nuevo y el resultado se almacena por separado.

El ciclo de vida de los datos en caché se controla con dos opciones:

- `staleTime`: tiempo, en milisegundos, durante el cual los datos se consideran frescos (fresh). Mientras no vence, la query entrega los datos de la caché sin volver a consultar el servidor. Su valor por defecto es 0, por lo que los datos se consideran obsoletos (stale) apenas se obtienen.
- `gcTime`: tiempo, en milisegundos, durante el cual los datos de una query inactiva (sin componentes montados que la usen) permanecen en la caché antes de eliminarse. Su valor por defecto es de 5 minutos. En TanStack Query v4 esta opción se llamaba `cacheTime`.

```
const { data, isLoading, isError } = useQuery<Product[]>({
  queryKey: ["products"],
  queryFn: getProducts,
  staleTime: 1000 * 60,
  gcTime: 1000 * 60 * 10,
});
```

Con esta configuración, los productos se consideran frescos durante un minuto y, cuando ningún componente los usa, permanecen en la caché durante diez minutos. Ambas opciones también pueden definirse para toda la aplicación en las `defaultOptions` del `QueryClient`.

5. Beneficios y funcionamiento automático

TanStack Query transforma el flujo de trabajo de la siguiente manera:

- **Verificación de caché:** si ya existen datos para esa `queryKey`, se muestran de inmediato. Si además están obsoletos según `staleTime` (con el valor por defecto, 0, siempre lo están), se vuelven a pedir en segundo plano y la vista se actualiza cuando llega la respuesta.
- **Deduplicación:** si varios componentes usan la misma `queryKey` mientras hay una petición en curso, comparten esa única petición y reciben el mismo resultado.
- **Refetch en segundo plano:** actualiza los datos automáticamente cuando se monta un nuevo componente que usa la query, cuando el usuario vuelve a la pestaña o cuando se recupera la conexión, siempre que los datos estén obsoletos (stale). Estos comportamientos se controlan con las opciones `refetchOnMount`, `refetchOnWindowFocus` y `refetchOnReconnect`.
- **Menos código repetitivo (boilerplate):** elimina la necesidad de declarar a mano los estados de carga, error y datos, lo que hace al código más legible y mantenible.

Bibliografía

1. Microsoft. (s. f.). *TypeScript handbook*. https://www.typescriptlang.org/docs/handbook
2. React. (s. f.). *React documentation*. https://react.dev
3. TanStack. (s. f.). *TanStack documentation*. https://tanstack.com